

purescript-sigil: SVG Rendering Backend

Motivation

Sigil currently renders type class definitions, ADT declarations, and type signatures as **HTML** (`Sigil.Html`). This works well for standalone pages and HTML-via-HATS layouts, but breaks down when type information needs to live inside SVG — DAG nodes, force-directed graphs, hylograph canvases, or any context where the visual container is SVG.

The concrete trigger: the Counterexamples article wants sigil-rendered class definitions as DAG nodes, not beside them. Embedding HTML in SVG requires `<foreignObject>`, which has cross-browser quirks (sizing, overflow, hit-testing). A native SVG renderer avoids all of this.

Design

Parallel to Sigil.Html

The existing library structure:

```
Sigil.Types      -- RenderType, SuperclassInfo, Constraint, etc.
Sigil.Parse      -- parseToRenderType (CST → RenderType)
Sigil.Color      -- type variable color assignment
Sigil.Html       -- HTML rendering (renderClassDeclInto,
renderSignatureInto, etc.)
Sigil.Svg.Layout -- SVG layout computation (already exists for
sparklines)
Sigil.Svg.Emit   -- SVG string emission (already exists)
```

The SVG backend adds:

```
Sigil.Svg.Layout.ClassDef -- already exists (layout computation)
Sigil.Svg.Layout.Signature -- already exists
Sigil.Svg.Layout.ADT      -- already exists
Sigil.Svg.Layout.Siglet   -- already exists
```

What's missing is **DOM-targeted SVG rendering** — the equivalent of `renderClassDeclInto` but producing SVG elements instead of HTML elements. The existing `Sigil.Svg.*` modules compute layouts and emit SVG strings; we need functions that inject into live DOM (like the HTML renderer does).

API Surface

```
-- Mirror of Sigil.Html API but for SVG contexts
module Sigil.Svg

renderClassDeclSvg
```

```

:: { name :: String
    , typeParams :: Array String
    , superclasses :: Array SuperclassInfo
    , methods :: Array { name :: String, ast :: Maybe RenderType }
    }
-> Tree    -- HATS Tree (SVG elements)

renderSignatureSvg
:: String          -- name
-> Maybe RenderType -- parsed type
-> Tree

renderAdtSvg
:: { name :: String
    , typeParams :: Array String
    , constructors :: Array { ... }
    }
-> Tree

```

Key difference from HTML: returns a HATS **Tree** rather than performing **Effect Unit** injection. This lets the caller compose it directly into a HATS SVG tree — as a DAG node child, a tooltip body, a force graph label, etc.

Alternative: renderInto for SVG

Also provide the imperative **renderInto** variant for cases where the SVG container already exists:

```
renderClassDeclIntoSvg :: String -> ClassDeclInput -> Effect Unit
```

HATS Integration

The SVG renderer should produce HATS **Tree** values using the existing **elem**, **staticStr**, etc. constructors. This means:

- Text → **elem Text** with **textContent**
- Colored spans → **elem Text** with **fill** attribute
- Grouping → **elem Group** with **transform**
- Boxes → **elem Rect** with **fill**, **stroke**, **rx**
- Arrow chains → **elem Text** segments or **elem Path** for arrow glyphs

Layout Engine

The existing **Sigil.Svg.Layout.*** modules already compute bounding boxes, positions, and sizes for SVG rendering. The new renderer builds on these:

1. **layoutClassDef** computes the geometry (widths, heights, positions of each method, header, etc.)
2. The new renderer takes that layout and produces HATS **Tree** nodes

Sizing Challenge

SVG text doesn't reflow. The layout engine needs to know text widths upfront. Current approach in `Sigil.Svg.Layout`: character-width estimation (`charWidth` parameter, typically ~7.8px for 13px monospace). This is approximate but works well enough for fixed-width fonts.

For variable-width content (class names in bold, type variables in *italic*), we may need a measurement pass or conservative estimates per font-weight/style.

Implementation Steps

Step 1: HATS Tree Builder for Existing Layouts

Wire up `Sigil.Svg.Layout.ClassDef` output to produce HATS `Tree` instead of raw SVG strings.

Step 2: Signature and ADT Renderers

Same treatment for `Sigil.Svg.Layout.Signature` and `Sigil.Svg.Layout.ADT`.

Step 3: Color and Style Consistency

Ensure SVG output uses the same color palette as HTML (indigo headers for classes, amber for ADTs, gold for type aliases, variable colors via `Sigil.Color`).

Step 4: renderInto Variants

Add DOM-injection functions that find an SVG element by selector and append the rendered tree.

Step 5: Publish

Publish as part of sigil 0.3.0 (or as a separate `sigil-svg` package if the dependency on hylograph-selection is unwanted in the core sigil library).

Dependency Considerations

- `Sigil.Html` has no dependency on hylograph — it emits raw HTML strings via FFI
- `Sigil.Svg.Emit` similarly emits SVG strings
- A HATS-returning renderer would depend on `hylograph-selection` (for `Tree`, `elem`, `ElementType`)
- **Decision:** keep the HATS integration in a separate module (`Sigil.Hats` or `Sigil.Svg.Hats`) so the core library stays dependency-light

Lessons from the Counterexamples Rewrite

1. **siblings creates <g> wrappers** — fine in SVG, breaks in HTML. An SVG-native renderer avoids this entirely.
2. **Two-phase render works** but adds complexity. If the renderer returns `Tree`, it composes directly into the HATS tree — single phase.
3. **foreignObject is fragile** — sizing, scrollbar, hit-testing issues across browsers. Native SVG avoids all of this.
4. **Text measurement** is the hard part. The HTML renderer gets it for free (browser reflow). SVG needs upfront computation.

Success Criteria

- `renderClassDeclSvg` produces a self-contained SVG subtree that can be embedded in any HATS SVG context
- Visual parity with HTML renderer (same colors, typography feel, information density)
- Published to PureScript registry as sigil 0.3.0
- Used in at least one showcase (Counterexamples DAG nodes or Ecosystem site)